Cloud Computing — Student Lab Manual

Oct 6, 2026 · @JP

How to use this manual

You will build seven labs on your own AWS account across three days, ending with a secure, auto-scaling web application. Each lab lists its goal, the concepts it teaches, step-by-step instructions, what you should see, a deliverable, and clean-up.

Lab	Day	Time	You will build	Reused later?
Lab 0	Before Day 1	30 min	A safe AWS account with MFA and a budget alert	Yes, all labs
Lab 1	Day 1	30 min	An EC2 virtual machine with a Docker container inside	No
Lab 2	Day 2	30 min	A static website hosted on S3	Yes, Lab 5
Lab 3	Day 2	35 min	A web server inside a VPC you design	Yes, VPC in Lab 6
Lab 4	Day 2	25 min	A serverless API with Lambda and DynamoDB	Yes, Lab 5
Lab 5	Day 3	30 min	Least-privilege IAM and an audit trail	No
Lab 6	Day 3	50 min	A load-balanced, auto-scaling web tier	No

Ground rules

- Work in **one region for the whole course**, the one your instructor names (for example Asia Pacific (Mumbai), ap-south-1). Check the region selector at the top-right of the console before every lab.
- Replace `yourname` in every resource name with your own name, for example `lab1-priya`. This keeps resources easy to find and clean up.
- Text in `monospace` is something you type or paste exactly.

- Do the clean-up step at the end of each lab. Forgotten resources are the most common cause of surprise bills.
- Stuck for more than 5 minutes? Check the Troubleshooting section at the end, then ask a neighbour or the instructor.

Cost safety. Every lab is designed to fit inside AWS's free allowances and new-account credits. The one paid-by-the-hour item is the load balancer in Lab 6, so finish Lab 6's clean-up the same day.

Submitting work. Collect each lab's deliverable screenshot in one document and submit it as a single PDF at the end of Day 3.

Lab 0 — Set up a safe AWS account (before Day 1)

Goal: an AWS account that is protected by MFA, warns you before you spend money, and is set to the class region. Do this at home before Day 1; sign-up can take 15–30 minutes.

Concepts: root user, multi-factor authentication (MFA), budgets, regions.

Step 1 — Create the account

1. Go to aws.amazon.com/free and choose **Create a Free Account**.
2. Enter your email, an account name (for example `cc-course-yourname`) and verify the email code.
3. Set a strong root password and store it safely.
4. Choose the **Free plan** when asked to pick a plan. New accounts get $100 in credits at sign-up and can earn up to $100 more by completing onboarding tasks. The Free plan lasts 6 months or until the credits are used, and does not bill you.
5. Enter contact details, a payment card (used for identity verification), and complete phone verification.
6. Sign in to the **AWS Management Console** as the root user.

Step 2 — Turn on MFA for the root user

1. Click your account name (top-right) → **Security credentials**.
2. Under **Multi-factor authentication (MFA)**, choose **Assign MFA device**.
3. Name it `root-phone`, pick **Authenticator app**, and scan the QR code with Google Authenticator, Microsoft Authenticator or similar.
4. Enter two consecutive codes and choose **Add MFA**.

Step 3 — Create a budget alert

1. Search for **Billing and Cost Management** in the top search bar and open it.
2. Choose **Budgets** → **Create budget** → **Use a template** → **Monthly cost budget**.
3. Budget name `course-budget`, amount `5` (USD), and your email as the recipient.
4. Choose **Create budget**. You will now get an email when spending is forecast to cross the amount.
5. Open **Credits** in the same console and note your credit balance.

Step 4 — Set the class region

1. Click the region name at the top-right of the console.
2. Select the region your instructor names, for example **Asia Pacific (Mumbai) ap-south-1**.

What you should see: MFA shows as assigned, a budget named `course-budget` exists, and the console shows the class region.

Deliverable: a screenshot of the Budgets page.

Why it matters: the root user can do anything, including close the account. MFA stops someone with your password from logging in, and the budget alert is your early warning against forgotten resources.

Lab 1 — Your first virtual machine, and a container inside it (Day 1, 30 min)

Goal: launch an EC2 instance, inspect the virtual hardware the hypervisor gives it, and run a Docker container inside it.

Concepts: virtual machine, hypervisor, AMI, instance type, security group, OS-level virtualization (containers).

Step 1 — Launch the instance

1. In the console search bar type `EC2` and open it. Confirm the class region at the top-right.
2. Choose **Launch instance**.
3. **Name:** `lab1-yourname`.

4. **Application and OS Images (AMI):** keep **Amazon Linux 2023**. An AMI is the template disk image the VM boots from.
5. **Instance type:** pick one marked **Free tier eligible**, for example `t3.micro` (2 vCPUs, 1 GiB memory).
6. **Key pair:** choose **Proceed without a key pair**. You will connect through the browser instead.
7. **Network settings:** keep the default VPC and subnet. Leave **Allow SSH traffic from Anywhere** ticked. (On Day 3 you will learn why this is risky in real systems.)
8. Keep the default storage and choose **Launch instance**, then **View all instances**.

Step 2 — Connect

1. Wait until **Instance state** is *Running* and **Status check** shows *2/2 checks passed* (about 1–2 minutes; use the refresh button).
2. Select the instance → **Connect** → **EC2 Instance Connect** tab → **Connect**. A Linux terminal opens in a new browser tab.

Step 3 — Inspect the virtual hardware

Run these commands one at a time and note the answers:

```
systemd-detect-virt                          # which hypervisor runs this VM?
nproc                                        # how many virtual CPUs?
lscpu | grep -E 'Model name|Hypervisor'   # physical CPU model and hypervisor
vendor
free -h                                      # how much memory?
df -h /                                      # how big is the virtual disk?
```

What you should see: `systemd-detect-virt` prints `amazon` — the AWS Nitro hypervisor, which is based on KVM. On `t3.micro`, `nproc` prints `2` and `free -h` shows about 1 GiB of memory. None of this hardware is a physical machine dedicated to you; it is a slice of a shared server.

Step 4 — Run a container inside the VM

```
sudo dnf install -y docker                   # install the Docker engine
sudo systemctl start docker                  # start it
time sudo docker run --rm alpine echo hi     # start a container, print 'hi',
remove it
```

What you should see: Docker downloads the small `alpine` image, prints `hi`, and `time` reports the duration. Run the last command a second time: it now finishes in about a second, because the image is cached. Compare that with the minute or so your VM needed to boot.

Think about it: you now have two layers of virtualization — a hardware-level VM (EC2) and an OS-level container (Docker) that shares the VM's Linux kernel.

Step 5 — Clean up

1. Close the terminal tab.
2. In EC2 → **Instances**, select `lab1-yourname` → **Instance state** → **Terminate (delete) instance** → confirm.

Deliverable: one screenshot showing the outputs of `systemd-detect-virt` and `nproc`, and one showing the `docker run` output with its time.

Lab 2 — Host a static website on Amazon S3 (Day 2, 30 min)

Goal: publish a website with no server at all, using S3 object storage.

Concepts: object storage, buckets and objects, globally unique names, resource-based policies, Block Public Access.

Step 1 — Prepare two files

On your laptop, create `index.html` (your own page from the Day 1 homework is fine) and `error.html`. Minimal versions:

```
<!-- index.html -->
<h1>Hello from S3</h1>
<p>Built by yourname for the Cloud Computing course.</p>
```

```
<!-- error.html -->
<h1>Oops — page not found</h1>
```

Step 2 — Create the bucket

1. Open **S3** → **Create bucket**.
2. **Bucket name:** `cc-lab2-yourname-1234` (replace `1234` with any digits). Names must be unique across all AWS accounts worldwide, lowercase, with no spaces.

3. Confirm the region is the class region. Keep all other defaults, including **Block all public access** ticked for now.
4. Choose **Create bucket**.

Step 3 — Upload the files

1. Open your bucket → **Upload** → **Add files** → select `index.html` and `error.html` → **Upload**.

Step 4 — Turn on static website hosting

1. Open the **Properties** tab and scroll to the bottom: **Static website hosting** → **Edit**.
2. Choose **Enable**, hosting type **Host a static website**.
3. **Index document:** `index.html`. **Error document:** `error.html`. Choose **Save changes**.
4. Scroll down again and copy the **Bucket website endpoint** URL.

Open that URL now. You get **403 Forbidden** — the bucket is still private. That is S3 protecting you by default.

Step 5 — Allow public reads

1. Open the **Permissions** tab → **Block public access (bucket settings)** → **Edit**.
2. Untick **Block all public access** → **Save changes** → type `confirm`.
3. Still on **Permissions**, scroll to **Bucket policy** → **Edit**, and paste this policy, replacing `YOUR-BUCKET` with your bucket name:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "PublicRead",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::YOUR-BUCKET/*"
  }]
}
```

4. Choose **Save changes**. The bucket now shows a red **Publicly accessible** label.

Read the policy: *Effect* Allow, *Principal* everyone (`*`), *Action* read an object (`s3:GetObject`), *Resource* every object in your bucket (`/*`). It allows reading only — nobody else can upload or delete.

Step 6 — Test

1. Refresh the website endpoint: your page appears.
2. Add `/nope` to the end of the URL: your `error.html` appears.

Bonus — versioning

1. **Properties → Bucket Versioning → Edit → Enable**.
2. Change `index.html` on your laptop and upload it again.
3. Open the bucket, turn on **Show versions**, and see both versions. Delete the newest version to roll back.

Clean-up

Keep this bucket — you will lock it down in Lab 5.

Deliverable: your website URL, plus a screenshot of the page in the browser.

Lab 3 — A web server in a VPC you built (Day 2, 35 min)

Goal: design your own private network (VPC) across two Availability Zones and run a web server in its public subnet.

Concepts: VPC, CIDR blocks, public and private subnets, internet gateway, route tables, security groups, user data.

Step 1 — Create the VPC

1. Open **VPC → Create VPC**.
2. Choose **VPC and more** (the wizard builds the subnets and routes for you).
3. **Name tag auto-generation:** `lab3`.
4. **IPv4 CIDR block:** `10.0.0.0/16` (65,536 addresses).
5. **Number of Availability Zones:** `2`. **Public subnets:** `2`. **Private subnets:** `2`.
6. **NAT gateways:** `None` (NAT gateways are billed by the hour). **VPC endpoints:** `None`.
7. Choose **Create VPC** and wait for every line to turn green → **View VPC**.

8. Open the **Resource map** tab. Trace how the public subnets route to the internet gateway (`lab3-igw`) and the private subnets do not.

Step 2 — Launch the web server

1. Open **EC2** → **Launch instance**. **Name:** `lab3-web-yourname` .
2. **AMI:** Amazon Linux 2023. **Instance type:** a Free tier eligible type such as `t3.micro` . **Key pair:** Proceed without a key pair.
3. **Network settings** → **Edit**:
   - **VPC:** `lab3-vpc` .
   - **Subnet:** one whose name contains `public1` .
   - **Auto-assign public IP: Enable**.
   - **Firewall (security groups):** Create security group, name `web-sg` .
   - Remove the SSH rule. Choose **Add security group rule**: Type **HTTP**, Source type **Anywhere**.
4. Expand **Advanced details**, scroll to **User data**, and paste this script. It runs once, as root, the first time the instance boots:

```
#!/bin/bash
dnf install -y nginx
echo "<h1>Hello from $(hostname)</h1>" > /usr/share/nginx/html/index.html
systemctl enable --now nginx
```

5. Choose **Launch instance**.

Step 3 — Test

1. Wait 2–3 minutes after the instance shows *Running* (nginx is installing).
2. Copy the instance's **Public IPv4 address** and open `http://` followed by that address. Type `http://` yourself — browsers often try `https://` , which will fail.

What you should see: "Hello from ip-10-0-…", showing the instance's private hostname inside your 10.0.0.0/16 network.

Step 4 — Explore the routing

1. In **VPC** → **Route tables**, open the route table of a public subnet. Find the route `0.0.0.0/0` → `igw-…` .
2. Open the route table of a private subnet. It has only the `local` route.

Challenge

Launch a second instance with the same settings but in a **private** subnet. Can you open it in your browser? Write one sentence explaining why, using the route tables you just looked at. Terminate it afterwards.

Clean-up

1. Terminate `lab3-web-yourname` (and any challenge instance).
2. **Keep the VPC** `lab3-vpc` — you will reuse it in Lab 6. A VPC with no NAT gateway costs nothing.

Deliverable: a screenshot of your page in the browser, and a screenshot of the VPC resource map.

Lab 4 — A serverless visitor counter (Day 2, 25 min)

Goal: build a public web API that counts visits, with no server to manage, using Lambda and DynamoDB.

Concepts: serverless, functions as a service, event-driven code, NoSQL key-value tables, execution roles, CloudWatch Logs.

Step 1 — Create the table

1. Open **DynamoDB** → **Tables** → **Create table**.
2. **Table name:** `visits`. **Partition key:** `page`, type **String**.
3. Keep **Default settings** → **Create table**. Wait until the status is *Active*.

A partition key is the unique ID for each item — here one item per page, such as `home`.

Step 2 — Create the function

1. Open **Lambda** → **Create function** → **Author from scratch**.
2. **Function name:** `visit-counter`. **Runtime: Python 3.13**. Keep the default architecture.
3. Under **Permissions**, keep **Create a new role with basic Lambda permissions**. This role lets the function write logs, and nothing else yet.
4. Choose **Create function**.

Step 3 — Add the code

On the **Code** tab, replace everything in `lambda_function.py` with:

```
import json, boto3

# Connect to the 'visits' table once, when the function starts
table = boto3.resource("dynamodb").Table("visits")

def lambda_handler(event, context):
    # Add 1 to 'hits' for the 'home' item (creates it the first time)
    r = table.update_item(
        Key={"page": "home"},
        UpdateExpression="ADD hits :one",
        ExpressionAttributeValues={":one": 1},
        ReturnValues="UPDATED_NEW")
    hits = int(r["Attributes"]["hits"])
    return {"statusCode": 200,
            "body": json.dumps({"visits": hits})}
```

Choose **Deploy**.

Step 4 — Give the function access to the table

1. Open the **Configuration** tab → **Permissions**, and click the **Role name** link (it opens IAM).
2. **Add permissions** → **Attach policies** → search `AmazonDynamoDBFullAccess` → tick it → **Add permissions**.

This policy is far broader than the function needs. That is deliberate: in Lab 5 you will replace it with a least-privilege policy.

Step 5 — Give the function a URL

1. Back in Lambda: **Configuration** → **Function URL** → **Create function URL**.
2. **Auth type:** `NONE` → **Save**.
3. Open the **Function URL**. Refresh a few times.

What you should see: `{"visits": 1}`, then 2, 3, and so on. Share your URL with a neighbour and watch your count rise. In DynamoDB → **Explore items** → `visits`, the `home` item shows the same number.

Step 6 — Read the logs

1. In Lambda, open **Monitor** → **View CloudWatch logs**.
2. Open the newest log stream. Each visit shows a `START`, `END` and `REPORT` line with the duration and memory used — this is what you pay for, per millisecond.

Clean-up

Keep the function and table for Lab 5. Lambda and DynamoDB have always-free monthly allowances, so this lab should cost nothing.

Deliverable: a screenshot of your Function URL showing a count above 5, and one of the CloudWatch log stream.

Lab 5 — Least privilege and an audit trail (Day 3, 30 min)

Goal: give a person and a program only the permissions they need, prove that everything else is denied, and find the evidence in the audit log.

Concepts: IAM users, groups, roles and policies; least privilege; managed vs inline policies; CloudTrail; Block Public Access.

Part A — A read-only colleague

1. Open **IAM** → **User groups** → **Create group**. **Name:** `s3-readers`. Under **Attach permissions policies**, tick `AmazonS3ReadOnlyAccess` → **Create user group**.
2. **IAM** → **Users** → **Create user**. **User name:** `dev-yourname`.
3. Tick **Provide user access to the AWS Management Console** → **I want to create an IAM user**. Choose **Custom password**, set one, and untick **Users must create a new password at next sign-in** → **Next**.
4. **Add user to group:** tick `s3-readers` → **Next** → **Create user**.
5. Copy the **Console sign-in URL** shown on the final page (it contains your 12-digit account ID).
6. Open a **private/incognito browser window**, go to that URL and sign in as `dev-yourname`.
7. As `dev-yourname`, open **S3** and your Lab 2 bucket. Listing and opening objects works.
8. Now try two things that need write access:
   - Select an object → **Delete**. You get **Access Denied** (or a failed delete).

- Choose **Create bucket**, any name → **Create**. You get **Access Denied**.

The user can do exactly what the group's policy allows and nothing else. Close the private window.

Part B — A least-privilege Lambda role

Back in your own (root) session:

1. Open **Lambda** → `visit-counter` → **Configuration** → **Permissions** → click the role name.
2. In the role's **Permissions policies**, tick `AmazonDynamoDBFullAccess` → **Remove** → confirm.
3. Refresh your Function URL. It now fails with an error: the function has lost access.
4. Open **DynamoDB** → **Tables** → `visits` → **Overview** → **General information**, and copy the **Amazon Resource Name (ARN)**.
5. Back on the role: **Add permissions** → **Create inline policy** → **JSON**, and paste this, replacing `TABLE-ARN` with the ARN you copied:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": "dynamodb:UpdateItem",
    "Resource": "TABLE-ARN"
  }]
}
```

6. **Next** → **Policy name:** `visits-update-only` → **Create policy**.
7. Refresh the Function URL. The count works again.

Compare: before, the function could read, change or delete *every* DynamoDB table in your account. Now it can do one action on one table. If its code were ever compromised, the damage is limited to adding to a counter.

Part C — Find the evidence in CloudTrail

1. Open **CloudTrail** → **Event history** (no setup needed; it keeps 90 days of management events).
2. Set the filter to **User name** = `dev-yourname`. Events can take up to about 5 minutes to appear.

3. Find the **CreateBucket** event and open it. In the event record, find `errorCode: AccessDenied`, the time, and the source IP address.

You will not find the denied object delete here. Object reads and deletes are *data events*, which CloudTrail records only if you create a trail that logs them. Bucket creation is a *management event*, which Event history always shows.

Part D — Make the Lab 2 bucket private again

1. Open your Lab 2 bucket → **Permissions** → **Block public access** → **Edit** → tick **Block all public access** → **Save** → type `confirm`.
2. Refresh your website URL: **403 Forbidden**.

Block Public Access overrides the public bucket policy. In real projects you keep it on and serve sites through CloudFront with Origin Access Control instead (covered in Unit 5).

Clean-up

Keep everything until the final clean-up at the end of Day 3.

Deliverable: a screenshot of the Access Denied message as `dev-yourname`, and a screenshot of the CreateBucket event showing `AccessDenied` in CloudTrail.

Lab 6 — Capstone: a load-balanced, auto-scaling web tier (Day 3, 50 min)

Goal: build the centre of the ShopKart case-study architecture — an Application Load Balancer in front of an Auto Scaling group spread over two Availability Zones — then prove that it balances traffic, heals itself and scales.

Concepts: launch templates, target groups, Application Load Balancer, Auto Scaling groups, target-tracking policies, health checks, security-group referencing, instance metadata (IMDSv2).

Build order matters: security groups → launch template → target group → load balancer → Auto Scaling group. Everything goes in `lab3-vpc` from Lab 3.

Part A — Build (30 min)

Step 1 — Two security groups

1. **EC2 → Security Groups → Create security group**. **Name:** `alb-sg`. **VPC:** `lab3-vpc`. Inbound rule: **HTTP**, source **Anywhere-IPv4**. **Create**.
2. Create another: **Name:** `app-sg`, **VPC:** `lab3-vpc`. Inbound rule: **HTTP**, source **Custom**, then pick `alb-sg` from the list. **Create**.

The instances will accept web traffic *only* from the load balancer, never directly from the internet.

Step 2 — Launch template

1. **EC2 → Launch Templates → Create launch template**. **Name:** `lt-shopkart`.
2. **AMI:** Amazon Linux 2023. **Instance type:** a Free tier eligible type such as `t3.micro`. **Key pair:** Don't include in launch template.
3. **Network settings:** leave **Subnet** as *Don't include*. Expand **Advanced network configuration → Add network interface**: **Auto-assign public IP** = **Enable**, **Security groups** = `app-sg`.
4. **Advanced details → User data**, paste:

```
#!/bin/bash
dnf install -y nginx
M=http://169.254.169.254/latest
H=X-aws-ec2-metadata-token
# IMDSv2: get a session token, then ask which AZ this instance is in
T=$(curl -sX PUT $M/api/token -H "$H-ttl-seconds: 300")
AZ=$(curl -s -H "$H: $T" $M/meta-data/placement/availability-zone)
echo "<h1>ShopKart · $(hostname) · $AZ</h1>" >
/usr/share/nginx/html/index.html
systemctl enable --now nginx
```

5. **Create launch template**.

Step 3 — Target group

1. **EC2 → Target Groups → Create target group**. **Target type:** Instances. **Name:** `tg-web`. **Protocol/Port:** HTTP 80. **VPC:** `lab3-vpc`. Health check path: `/`.
2. **Next** → register nothing → **Create target group**. The Auto Scaling group will register instances for you.

Step 4 — Application Load Balancer

1. **EC2 → Load Balancers → Create load balancer → Application Load Balancer → Create**.
2. **Name:** `alb-shopkart`. **Scheme:** Internet-facing. **IP address type:** IPv4.

3. **VPC:** `lab3-vpc`. **Mappings:** tick both Availability Zones and choose the **public** subnet in each.
4. **Security groups:** remove `default`, select `alb-sg`.
5. **Listeners:** HTTP 80 → forward to `tg-web`.
6. **Create load balancer**. Provisioning takes a few minutes; carry on.

Step 5 — Auto Scaling group

1. **EC2 → Auto Scaling Groups → Create Auto Scaling group**. **Name:** `asg-shopkart`. **Launch template:** `lt-shopkart` → **Next**.
2. **VPC:** `lab3-vpc`. **Availability Zones and subnets:** both **public** subnets → **Next**.
3. **Load balancing: Attach to an existing load balancer** → choose `tg-web` from the target groups. Tick **Turn on Elastic Load Balancing health checks** → **Next**.
4. **Group size:** Desired `2`, Min `2`, Max `4`.
5. **Automatic scaling: Target tracking scaling policy**, metric **Average CPU utilization**, target value `50` → **Next** → skip notifications and tags → **Create Auto Scaling group**.
6. Open **EC2 → Target Groups →** `tg-web` **→ Targets**. Wait until both instances show **healthy** (3–5 minutes).

Part B — Prove it works (20 min)

Test 1 — Load balancing. Copy the load balancer's **DNS name** (EC2 → Load Balancers) and open it with `http://`. Refresh several times: the hostname and Availability Zone change. Or open **CloudShell** (the terminal icon in the console's top bar) and run:

```
for i in $(seq 10); do curl -s http://ALB-DNS-NAME; echo; done
```

Test 2 — Self-healing. In **EC2 → Instances**, terminate one of the two ShopKart instances. Open **Auto Scaling Groups →** `asg-shopkart` **→ Activity**: within a few minutes the group launches a replacement to get back to 2. The website stays up the whole time, served by the surviving instance.

Test 3 — Auto scaling.

1. Temporarily allow a browser SSH connection: edit `app-sg` → add inbound rule **SSH**, source **Anywhere-IPv4**.
2. Connect to one instance with **EC2 Instance Connect** and burn both vCPUs:

```
yes > /dev/null &      # fills CPU 1
yes > /dev/null &      # fills CPU 2
top                    # CPU near 100%; press q to quit
```

3. Open **CloudWatch → Alarms**. Within about 5–10 minutes the `TargetTracking-…-AlarmHigh` alarm goes into *In alarm*, and the Auto Scaling group's **Activity** tab shows new instances launching.
4. Stop the load with `pkill yes`, then **remove the SSH rule** from `app-sg`. After a while the AlarmLow alarm fires and the group scales back in to 2.

Opening SSH to the internet is a temporary shortcut for this test. In real systems you would use Systems Manager Session Manager, which needs no open inbound port.

Extension (optional) — CloudFront. CloudFront → Create distribution. **Origin:** your load balancer, protocol **HTTP only**. Keep other defaults, and decline optional paid extras such as WAF protections. After it deploys (a few minutes), open the `…cloudfront.net` domain: your site, now served over HTTPS from an edge location.

Deliverable: a screenshot of two responses showing different Availability Zones, and a screenshot of the Auto Scaling group's Activity history showing the replacement and scale-out launches.

Clean-up: do the final clean-up below today — the load balancer is billed by the hour.

Final clean-up (end of Day 3, 15 min)

Delete in this order — some resources block others from being deleted, and the Auto Scaling group relaunches any instance you delete before it. Tick each box as you go.

- [ ] **CloudFront distribution** (if you made one): select it → **Disable** → wait until it shows *Deployed* again → **Delete**.
- [ ] **Auto Scaling group** `asg-shopkart`: **Actions → Delete**. This also terminates its instances.
- [ ] **Load balancer** `alb-shopkart`: **Actions → Delete load balancer**.
- [ ] **Target group** `tg-web`: **Actions → Delete**.
- [ ] **Launch template** `lt-shopkart`: **Actions → Delete template**.
- [ ] **EC2 → Instances**: confirm none are *Running* or *Stopped*. **EC2 → Volumes**: delete any left *Available*.

- [ ] **VPC** `lab3-vpc` : **Actions** → **Delete VPC** (this removes its subnets, route tables, internet gateway and security groups). If it refuses, a load balancer or instance is still inside.
- [ ] **Lambda** `visit-counter` : **Actions** → **Delete**. Then in **IAM** → **Roles**, delete the role whose name starts with `visit-counter-role` .
- [ ] **DynamoDB** table `visits` : **Actions** → **Delete table**.
- [ ] **S3** bucket from Lab 2: **Empty** (type `permanently delete`) → then **Delete** (type the bucket name).
- [ ] **IAM**: delete user `dev-yourname` , then group `s3-readers` .
- [ ] **CloudWatch** → **Log groups**: delete `/aws/lambda/visit-counter` . **Alarms**: delete any left over.
- [ ] **Search everywhere: Resource Groups & Tag Editor** → **Tag Editor** → Regions *All regions*, Resource types *All supported resource types* → **Search resources**. Investigate anything you don't recognise.
- [ ] **Billing and Cost Management** → **Bills** and **Credits**: check again tomorrow that charges have stopped.

Keep your MFA and budget alert switched on for as long as you keep the account.

Troubleshooting quick reference

Most lab problems come from one of three causes: the wrong region, a missing security-group rule, or a missing permission. Check those first.

Lab	Symptom	Likely cause	Fix
Any	A resource you created has "disappeared"	Console is in a different region	Switch the region at the top-right back to the class region
1, 3	EC2 Instance Connect fails	No public IP, or no SSH rule in the security group	Use a public subnet with public IP enabled; allow SSH in the security group

Lab	Symptom	Likely cause	Fix
2	403 Forbidden on the website	Block Public Access still on, or wrong bucket name in the policy	Untick Block Public Access; check the policy's `Resource` ends in `YOUR-BUCKET/*`
2	404 Not Found	Index document name doesn't match the file	Names are case-sensitive: `index.html` in both places
3, 6	Page won't load in the browser	Browser forced `https://`, or nginx still installing	Type `http://` explicitly; wait 2–3 minutes after launch
3	Page won't load after waiting	Instance in a private subnet, or no HTTP rule	Relaunch in a `public` subnet; add HTTP to `web-sg`
4	URL shows "Internal Server Error"	Error inside the function	Open Monitor → CloudWatch logs and read the newest error
4	Log shows `AccessDeniedException`	Role has no DynamoDB permission	Attach the policy (Lab 4) or the inline policy (Lab 5)
4	Log shows `ResourceNotFoundException`	Table name or region mismatch	Table must be named `visits`, in the same region as the function
5	No events for `dev-yourname` in CloudTrail	Events still arriving	Wait about 5 minutes; check the time range and region
6	Targets stay *unhealthy*	nginx not running, or `app-sg` doesn't allow HTTP from `alb-sg`	Check the user data; check the `app-sg` inbound rule source

Lab	Symptom	Likely cause	Fix
6	Load balancer DNS returns 503	No healthy targets yet	Wait until both targets are healthy in `tg-web`
6	Instances have no public IP and never become healthy	Public IP not enabled in the launch template	Edit the template's network interface: auto-assign public IP = Enable
6	No scale-out after 10 minutes	Load not running, or still within the alarm period	Check `top` shows ~100% CPU; check CloudWatch → Alarms
Clean-up	VPC won't delete	Something is still inside it	Delete the load balancer and instances first, then retry